

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ Import Packages and Data

```
# Import the packages. You will need to add other packages to order to complete the assignment tasks
import pandas as pd
import numpy as np
import random
import io
from google.colab import files
from sklearn.preprocessing import StandardScaler
```

```
# Upload the MLData2026.csv file into your google drive.
# You may comment out (using #) the below code once the data has been imported
uploaded = files.upload()
MLData2026 = pd.read_csv(io.BytesIO(uploaded['MLData2026.csv']))
```

MLData2026.csv

MLData2026.csv(text/csv) - 49050823 bytes, last modified: 3/20/2026 - 100% done
Saving MLData2026.csv to MLData2026.csv

✓ Select Numeric Features

```
# Define the list of numeric features and the target variable column names
selected_columns = [
    'Assembled Payload Size',
```

```

'DYNRiskA Score',
'Response Size',
'Source Ping Time',
'Connection Rate',
'Server Response Packet Time',
'Packet Size',
'Packet TTL',
'Source IP Concurrent Connection',
'Class'
]

# Create mydata_numeric as a DataFrame using the selected columns from 'mydata'
data_numeric = MLData2026[selected_columns].copy() # Explicitly create a copy

#####
##### Add the relevant code in the space below to clean the numeric data #####
#####

data_numeric = data_numeric.replace(['-', ' '], np.nan)
for col in data_numeric.columns:
    data_numeric[col] = pd.to_numeric(data_numeric[col], errors='coerce')

dat_cleaned = data_numeric.dropna()

#####

dat_cleaned.shape
dat_cleaned.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 477627 entries, 0 to 502158
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Assembled Payload Size                477627 non-null int64
1   DYNRiskA Score                        477627 non-null float64
2   Response Size                         477627 non-null float64
3   Source Ping Time                      477627 non-null int64
4   Connection Rate                       477627 non-null float64
5   Server Response Packet Time           477627 non-null int64

```

```
6   Packet Size          477627 non-null   int64
7   Packet TTL           477627 non-null   int64
8   Source IP Concurrent Connection 477627 non-null   int64
9   Class                477627 non-null   int64
dtypes: float64(3), int64(7)
memory usage: 40.1 MB
```

✓ Create Unbalanced and Balanced Training Samples

```
# Separate samples of non-malicious and malicious events
dat_class0 = dat_cleaned[dat_cleaned['Class'] == 0] # non-malicious
dat_class1 = dat_cleaned[dat_cleaned['Class'] == 1] # malicious

n0 = 9800 # Number of Non-Malicious samples
n1 = 200  # Number of Malicious samples

# Randomly select the required samples from each class (without replacement)
# Replace 'random_state' pre-specified value with your student ID
rand_class0 = dat_class0.sample(n=n0, replace=False, random_state=10729304) # <-- Replace with your student ID
rand_class1 = dat_class1.sample(n=n1, replace=False, random_state=10729304) # <-- Replace with your student ID

# Get the remaining unselected samples
unselected_class0 = dat_class0.drop(rand_class0.index)
unselected_class1 = dat_class1.drop(rand_class1.index)

# Combine the selected samples to form the training set, which is unbalanced
mydata_train_unbln = pd.concat([rand_class0, rand_class1], axis=0)

# Upsample the minority class to be same size of the majority class
# to form a balanced training set
rand_class2 = dat_class1.sample(n=n0, replace=True, random_state=10729304) # <-- Replace with your student ID
mydata_train_bln = pd.concat([rand_class0, rand_class2], axis=0)

# Combine the unselected samples to form the test set
mydata_test = pd.concat([unselected_class0, unselected_class1], axis=0)
```

```
# Check dimension of sub-sample
print(f"\nDimension of unbalanced training data, {mydata_train_unbln.shape}")
print(f"Dimension of balanced training data, {mydata_train_bln.shape}")
print(f"Dimension of test samples: {mydata_test.shape}")
```

```
Dimension of unbalanced training data, (10000, 10)
Dimension of balanced training data, (19600, 10)
Dimension of test samples: (467577, 10)
```

✓ Create Unstandardised and Standardised Datasets

```
# Separate features (X) and target (y) for the unstandardised TEST set
X_test = mydata_test.iloc[:, :-1] # Features for prediction
y_test = mydata_test.iloc[:, -1]  # Target variable

#####

# Separate features (X) and target (y) for the unbalanced training data
X_unbln = mydata_train_unbln.iloc[:, :-1] # Features for prediction
y_unbln = mydata_train_unbln.iloc[:, -1]  # Target variable

# Standardise the features for both training and test sets
scaler = StandardScaler()
X_unbln_std = scaler.fit_transform(X_unbln)
X_test_unbln_std = scaler.transform(X_test)

#####

# Separate features (X) and target (y) for the balanced training data
X_bln = mydata_train_bln.iloc[:, :-1] # Features for prediction
y_bln = mydata_train_bln.iloc[:, -1]  # Target variable

# Standardise the features for both training and test sets
scaler = StandardScaler()
X_bln_std = scaler.fit_transform(X_bln)
X_test_bln_std = scaler.transform(X_test)
```

✓ Select ML Modelling Approaches

```
random.seed(10729304) # <- Change to your Student ID.

models_list1 = ["Logistic Ridge Regression", "Logistic LASSO Regression",
                "Logistic Elastic-Net Regression"]
models_list2 = ["Classification Tree", "Bagging Tree", "Random Forest"]

# Randomly select one model from each list
my_models = [random.choice(models_list1), random.choice(models_list2)]
```

```
# Convert the list to a DataFrame
my_models_df = pd.DataFrame(my_models, columns=['Selected Models'])
display(my_models_df)
```

	Selected Models 
0	Logistic Ridge Regression 
1	Random Forest

Next steps: [Generate code with my_models_df](#) [New interactive sheet](#)

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, RepeatedStratifiedKFold
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

```
# Define the CV strategy: 5-fold CV with 2 repeats
cv_strategy = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=10729304)

# Ridge Logistic Regression Setup
# Tune using 10 alpha values (using C as the inverse of alpha)
param_grid_lr = {'C': np.logspace(-4, 4, 10)}
logreg_model = LogisticRegression(penalty='l2', solver='liblinear', random_state=10729304)

# Random Forest Setup
# Tune n_estimators and max_features (3 values each)
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_features': ['sqrt', 'log2', None]
}
rf_model = RandomForestClassifier(random_state=10729304)

def run_tuning(model, param_grid, X_train, y_train):
    grid_search = GridSearchCV(model, param_grid, cv=cv_strategy, scoring='f1', n_jobs=-1)
```

```
grid_search.fit(X_train, y_train)
return grid_search.best_estimator_, grid_search.best_params_
```

```
# Initialize Logistic Regression with Ridge (L2) regularization
# Unbalanced & Unstandardised
best_lr_unbln, params_lr_unbln = run_tuning(logreg_model, param_grid_lr, X_unbln, y_unbln)

# Balanced & Unstandardised
best_lr_bln, params_lr_bln = run_tuning(logreg_model, param_grid_lr, X_bln, y_bln)

# Unbalanced & Standardised
best_lr_unbln_std, params_lr_unbln_std = run_tuning(logreg_model, param_grid_lr, X_unbln_std, y_unbln)

# Balanced & Standardised
best_lr_bln_std, params_lr_bln_std = run_tuning(logreg_model, param_grid_lr, X_bln_std, y_bln)
print("Logistic Ridge Regression models initialized and trained.")
```

Logistic Ridge Regression models initialized and trained.

```
from sklearn.ensemble import RandomForestClassifier

# Initialize Random Forest Classifier
# Unbalanced & Unstandardised
best_rf_unbln, params_rf_unbln = run_tuning(rf_model, param_grid_rf, X_unbln, y_unbln)

# Balanced & Unstandardised
best_rf_bln, params_rf_bln = run_tuning(rf_model, param_grid_rf, X_bln, y_bln)

# Unbalanced & Standardised
best_rf_unbln_std, params_rf_unbln_std = run_tuning(rf_model, param_grid_rf, X_unbln_std, y_unbln)

# Balanced & Standardised
best_rf_bln_std, params_rf_bln_std = run_tuning(rf_model, param_grid_rf, X_bln_std, y_bln)

print("Random Forest Classifier models initialized and trained.")
```

Random Forest Classifier models initialized and trained.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Helper function to evaluate and print metrics
def evaluate_and_print(model, X_test, y_test, model_name, config_name):
    y_pred = model.predict(X_test)

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Calculate confusion matrix for the report
    cm = confusion_matrix(y_test, y_pred)

    print(f"\n--- {model_name} ({config_name}) ---")
    print(f"Accuracy: {acc:.4f} | Precision: {prec:.4f} | Sensitivity (Recall): {rec:.4f} | F1-Score: {f1:.4f}")
    print("Confusion Matrix:\n", cm)

# --- EVALUATION: LOGISTIC RIDGE REGRESSION ---
# Ensure X_test_unbln_std and X_test_bln_std are defined/transformed correctly
evaluate_and_print(best_lr_unbln, X_test, y_test, "Logistic Ridge", "Unbalanced/Unstandardised")
evaluate_and_print(best_lr_bln, X_test, y_test, "Logistic Ridge", "Balanced/Unstandardised")
evaluate_and_print(best_lr_unbln_std, X_test_unbln_std, y_test, "Logistic Ridge", "Unbalanced/Standardised")
evaluate_and_print(best_lr_bln_std, X_test_bln_std, y_test, "Logistic Ridge", "Balanced/Standardised")

# --- EVALUATION: RANDOM FOREST ---
evaluate_and_print(best_rf_unbln, X_test, y_test, "Random Forest", "Unbalanced/Unstandardised")
evaluate_and_print(best_rf_bln, X_test, y_test, "Random Forest", "Balanced/Unstandardised")
evaluate_and_print(best_rf_unbln_std, X_test_unbln_std, y_test, "Random Forest", "Unbalanced/Standardised")
evaluate_and_print(best_rf_bln_std, X_test_bln_std, y_test, "Random Forest", "Balanced/Standardised")

--- Logistic Ridge (Unbalanced/Unstandardised) ---
Accuracy: 0.9972 | Precision: 0.7492 | Sensitivity (Recall): 0.7573 | F1-Score: 0.7532
Confusion Matrix:
```



```
[[464235    676]
 [    647   2019]]

--- Logistic Ridge (Balanced/Unstandardised) ---
Accuracy: 0.9704 | Precision: 0.1580 | Sensitivity (Recall): 0.9670 | F1-Score: 0.2716
Confusion Matrix:
[[451169   13742]
 [     88    2578]]

--- Logistic Ridge (Unbalanced/Standardised) ---
Accuracy: 0.9980 | Precision: 0.8316 | Sensitivity (Recall): 0.8095 | F1-Score: 0.8204
Confusion Matrix:
[[464474     437]
 [    508    2158]]

--- Logistic Ridge (Balanced/Standardised) ---
Accuracy: 0.9719 | Precision: 0.1654 | Sensitivity (Recall): 0.9715 | F1-Score: 0.2826
Confusion Matrix:
[[451839   13072]
 [     76    2590]]

--- Random Forest (Unbalanced/Unstandardised) ---
Accuracy: 0.9993 | Precision: 0.9434 | Sensitivity (Recall): 0.9370 | F1-Score: 0.9402
Confusion Matrix:
[[464761     150]
 [    168    2498]]

--- Random Forest (Balanced/Unstandardised) ---
Accuracy: 0.9967 | Precision: 0.6306 | Sensitivity (Recall): 1.0000 | F1-Score: 0.7734
Confusion Matrix:
[[463349    1562]
 [      0    2666]]

--- Random Forest (Unbalanced/Standardised) ---
Accuracy: 0.9993 | Precision: 0.9433 | Sensitivity (Recall): 0.9362 | F1-Score: 0.9398
Confusion Matrix:
[[464761     150]
 [    170    2496]]

--- Random Forest (Balanced/Standardised) ---
Accuracy: 0.9967 | Precision: 0.6306 | Sensitivity (Recall): 1.0000 | F1-Score: 0.7734
Confusion Matrix:
```

```
[[463349 1562]
 [      0 2666]]
```

✓ Performance Comparison

To easily compare the performance of all models, let's consolidate the metrics into a single DataFrame.

```
from sklearn.model_selection import RepeatedStratifiedKFold, GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import numpy as np

# 1. Setup CV Strategy (5-fold, 2 repeats)
cv_strategy = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=10729304)

# 2. Define Hyperparameter Grids
param_grid_lr = {'C': np.logspace(-4, 4, 10)}
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_features': ['sqrt', 'log2', None]
}

# 3. Define the Evaluation Function
def evaluate_full_model(model, X_test, y_test, model_label, config_label):
    y_pred = model.predict(X_test)

    # Calculate components
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    # Calculate metrics
    metrics = {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Sensitivity (Recall)": recall_score(y_test, y_pred),
        "Specificity": tn / (tn + fp),
        "F1-Score": f1_score(y_test, y_pred)
    }
```

```

print(f"\n>>> RESULTS: {model_label} ({config_label})")
for k, v in metrics.items(): print(f"{k}: {v:.4f}")
print(f"Confusion Matrix:\n{cm}")
return metrics

# 4. Execution Loop for all 8 scenarios
# Note: Ensure these variables (X_unbln, X_bln, X_unbln_std, etc.) are defined in your notebook
configs = [
    (X_unbln, X_test, "Unbalanced/Unstandardised"),
    (X_bln, X_test, "Balanced/Unstandardised"),
    (X_unbln_std, X_test_unbln_std, "Unbalanced/Standardised"),
    (X_bln_std, X_test_bln_std, "Balanced/Standardised")
]

# Tuning & Evaluating Logistic Ridge
print("--- TUNING LOGISTIC RIDGE ---")
for X_train, X_test_curr, label in configs:
    grid = GridSearchCV(LogisticRegression(penalty='l2', solver='liblinear', random_state=10729304),
                        param_grid_lr, cv=cv_strategy, scoring='f1', n_jobs=-1).fit(X_train, y_unbln if "Unbal
    evaluate_full_model(grid.best_estimator_, X_test_curr, y_test, "Logistic Ridge", label)
    print(f"Optimal Params: {grid.best_params_}")

# Tuning & Evaluating Random Forest
print("\n--- TUNING RANDOM FOREST ---")
for X_train, X_test_curr, label in configs:
    grid = GridSearchCV(RandomForestClassifier(random_state=10729304),
                        param_grid_rf, cv=cv_strategy, scoring='f1', n_jobs=-1).fit(X_train, y_unbln if "Unbal
    evaluate_full_model(grid.best_estimator_, X_test_curr, y_test, "Random Forest", label)
    print(f"Optimal Params: {grid.best_params_}")

```

```
CONFUSION MATRIX:  
[[451839 13072]  
 [    76  2590]]  
Optimal Params: {'C': np.float64(0.005994842503189409)}  
  
--- TUNING RANDOM FOREST ---  
  
>>> RESULTS: Random Forest (Unbalanced/Unstandardised)  
Accuracy: 0.9993
```

```
Precision: 0.6306  
Sensitivity (Recall): 1.0000  
Specificity: 0.9966  
F1-Score: 0.7734  
Confusion Matrix:  
[[463349  1562]  
 [      0  2666]]
```

```
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import RepeatedStratifiedKFold, GridSearchCV  
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
  
# 1. Setup CV Strategy (5-fold, 2 repeats)  
cv_strategy = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=10729304)  
  
# 2. Define Hyperparameter Grids  
param_grid_lr = {'C': np.logspace(-4, 4, 10)}  
param_grid_rf = {  
    'n_estimators': [50, 100, 200],  
    'max_features': ['sqrt', 'log2', None]  
}  
  
# 3. Define the Visual Evaluation Function  
def evaluate_and_plot_model(model, X_test, y_test, model_label, config_label):  
    y_pred = model.predict(X_test)  
  
    # Calculate components  
    cm = confusion_matrix(y_test, y_pred)  
    tn, fp, fn, tp = cm.ravel()  
  
    # Calculate metrics  
    metrics = {  
        "Accuracy": accuracy_score(y_test, y_pred),  
        "Precision": precision_score(y_test, y_pred),  
        "Sensitivity (Recall)": recall_score(y_test, y_pred),
```

```

        "Specificity": tn / (tn + fp),
        "F1-Score": f1_score(y_test, y_pred)
    }

    # Print Text Metrics
    print(f"\n>>> RESULTS: {model_label} ({config_label})")
    for k, v in metrics.items():
        print(f"    {k}: {v:.4f}")

    # Plot Visual Confusion Matrix
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['Predicted Benign', 'Predicted Attack'],
                yticklabels=['Actual Benign', 'Actual Attack'])
    plt.title(f"{model_label}\n({config_label})")
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    plt.show() # Displays the chart right inside Colab

    return metrics

# 4. Configuration Mappings
# Make sure these variables exist exactly like this from your previous cells
configs = [
    (X_unbln, X_test, "Unbalanced/Unstandardised"),
    (X_bln, X_test, "Balanced/Unstandardised"),
    (X_unbln_std, X_test_unbln_std, "Unbalanced/Standardised"),
    (X_bln_std, X_test_bln_std, "Balanced/Standardised")
]

# --- Run & Plot Logistic Ridge ---
print("--- TUNING & PLOTTING LOGISTIC RIDGE ---")
for X_train, X_test_curr, label in configs:
    grid = GridSearchCV(LogisticRegression(penalty='l2', solver='liblinear', random_state=10729304),
                        param_grid_lr, cv=cv_strategy, scoring='f1', n_jobs=-1).fit(X_train, y_unbln if "Unbal
    evaluate_and_plot_model(grid.best_estimator_, X_test_curr, y_test, "Logistic Ridge", label)

# --- Run & Plot Random Forest ---

```

```
print("\n--- TUNING & PLOTTING RANDOM FOREST ---")
for X_train, X_test_curr, label in configs:
    grid = GridSearchCV(RandomForestClassifier(random_state=10729304),
                        param_grid_rf, cv=cv_strategy, scoring='f1', n_jobs=-1).fit(X_train, y_unbln if "Unbal" in label else y_test)
    evaluate_and_plot_model(grid.best_estimator_, X_test_curr, y_test, "Random Forest", label)
```


--- TUNING & PLOTTING LOGISTIC RIDGE ---

>>> RESULTS: Logistic Ridge (Unbalanced/Unstandardised)

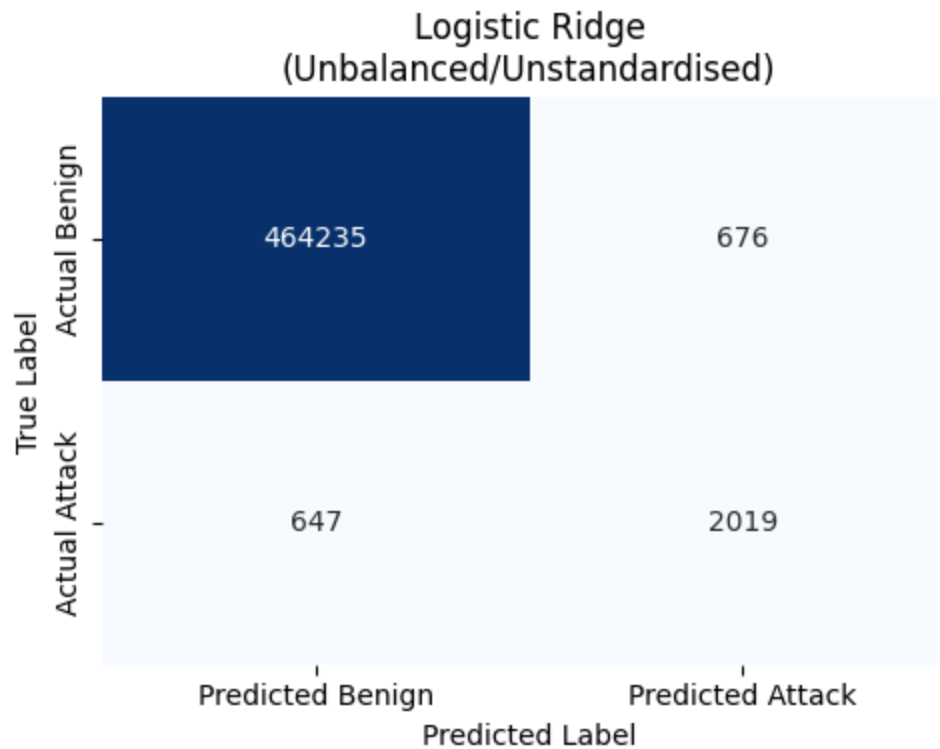
Accuracy: 0.9972

Precision: 0.7492

Sensitivity (Recall): 0.7573

Specificity: 0.9985

F1-Score: 0.7532



>>> RESULTS: Logistic Ridge (Balanced/Unstandardised)

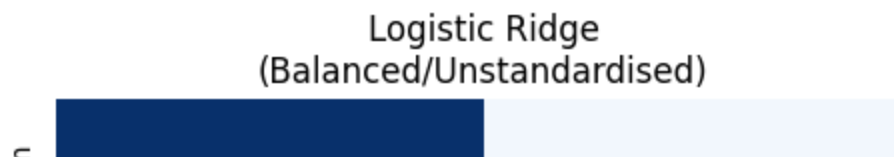
Accuracy: 0.9704

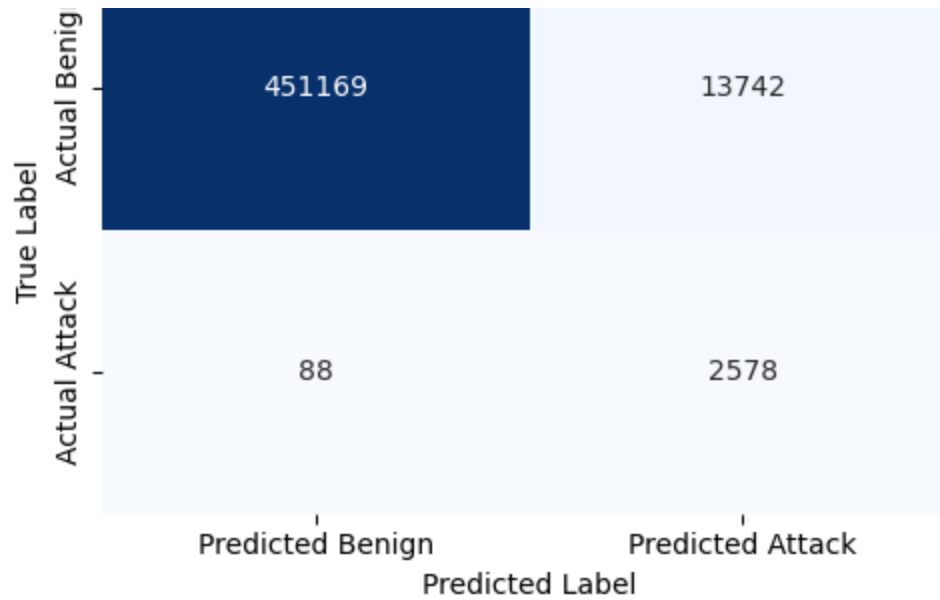
Precision: 0.1580

Sensitivity (Recall): 0.9670

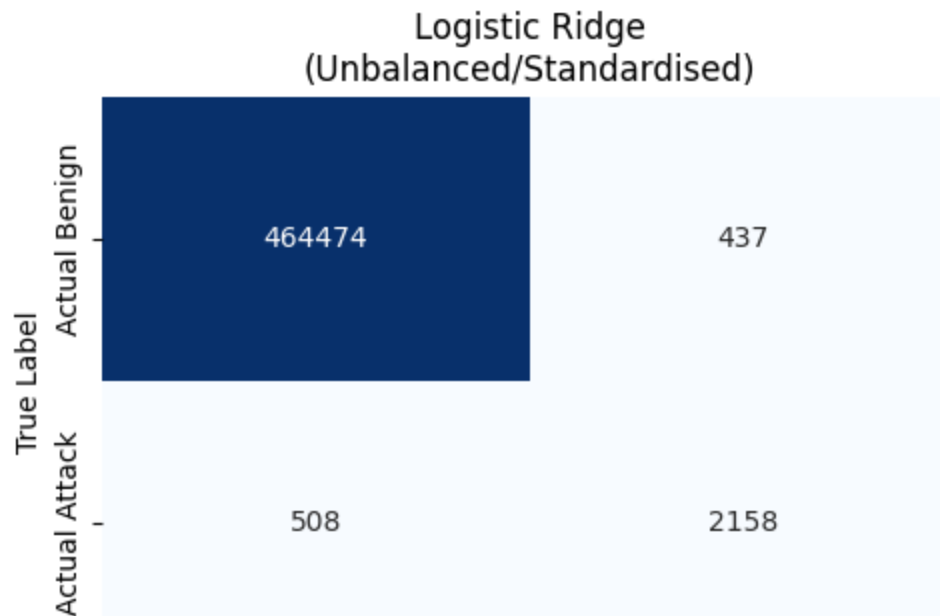
Specificity: 0.9704

F1-Score: 0.2716





```
>>> RESULTS: Logistic Ridge (Unbalanced/Standardised)
Accuracy: 0.9980
Precision: 0.8316
Sensitivity (Recall): 0.8095
Specificity: 0.9991
F1-Score: 0.8204
```



Predicted Benign Predicted Attack
Predicted Label

```
>>> RESULTS: Logistic Ridge (Balanced/Standardised)
```

```
Accuracy: 0.9719
```

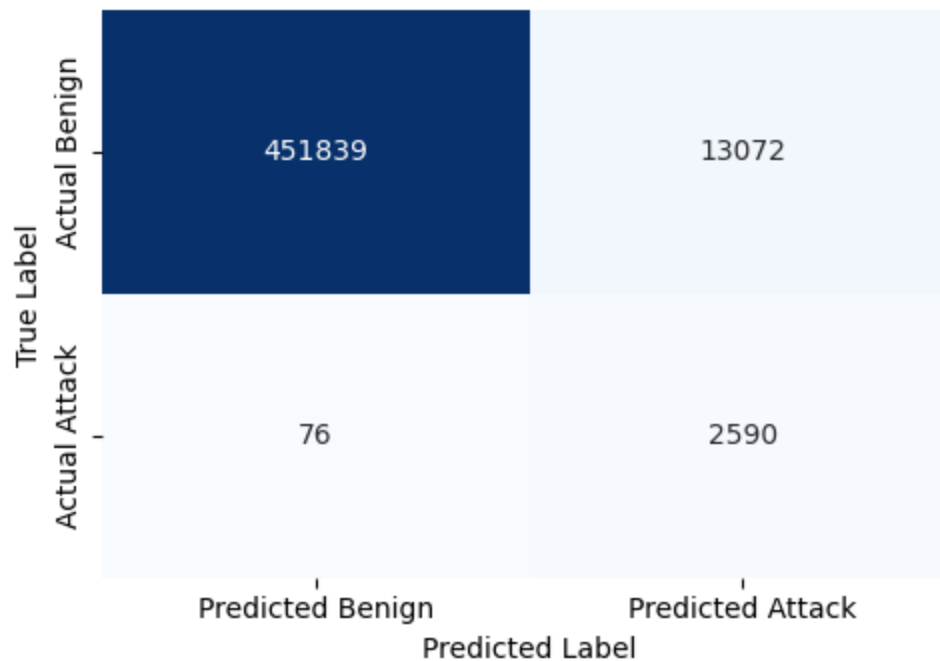
```
Precision: 0.1654
```

```
Sensitivity (Recall): 0.9715
```

```
Specificity: 0.9719
```

```
F1-Score: 0.2826
```

Logistic Ridge
(Balanced/Standardised)



```
--- TUNING & PLOTTING RANDOM FOREST ---
```

```
>>> RESULTS: Random Forest (Unbalanced/Unstandardised)
```

```
Accuracy: 0.9993
```

```
Precision: 0.9434
```

```
Sensitivity (Recall): 0.9370
```

```
Specificity: 0.9997
```

```
F1-Score: 0.9402
```

